How our model extracts motion

The complete walkthrough: pixels, crops, padding, Fourier correlation, motion evidence, prediction and memory

The core question is simple: **which small shift makes an earlier image patch line up with a later one?** The model asks this at 100 locations, over three temporal separations. It keeps a score for every candidate shift, allowing later computations to use both the preferred motion and the alternatives.

This LaTeX document follows the actual backbone used by the 19-task suite. This expanded edition was checked against the local implementation on October 17, 2026. It integrates the crop geometry, both padding stages, energy normalization, Fourier arithmetic and the correlation proof into one continuous explanation. The original numerical demonstrations use generated inputs on CPU; additional checks verify the Fourier identities in this edition.

Operation	What it contributes
Pixels and overlapping crops	A local light-dark pattern at each image location.
Mean subtraction and windowing	Emphasis on local structure and the crop interior.
81 normalized shift scores	Evidence about correspondence between two observations.
Learned refinement and softmax	A distribution over candidate displacements.
Three lags and a weighted mean	Short- and longer-interval motion, plus a compact velocity estimate.
Prediction and temporal comparison	Whether motion continues and how local evidence changes.
Shared spatial attention	Which location contributes to memory and the response.

How to read the equations

Every symbol is introduced where it first appears. A scalar is one number; a vector is an ordered list; a matrix is a table of numbers. A sum adds entries, an average divides their sum by their count, and a norm measures the size of a vector. Sections 1-8 explain the motion measurement itself. Sections 9-14 explain how it becomes useful to the task. Sections 15-21 provide checked examples, theory, a glossary, design settings and sources. Section 6 develops the Fourier calculation over several pages, beginning with the second padding stage.

Implementation anchors: SecondPass/StabilizedAttentionKrauzlis/model.py and SecondPass/MemoryGuidedSearch/train.py. The architecture and training-source hashes are recorded with the accompanying verification receipt.

Contents

Reading guide 1

1 Start with the actual input 3

2 Where the model looks 4

3 Prepare the crop 6

4 Ask 81 correspondence questions 7

5 Separate shape from signal strength 8

6 The FFT computes the same sums 10

7 Turn scores into a population 17

8 Three lags, one velocity convention 18

9 Use motion to predict pixels 19

10 Measure prediction error 20

11 Remember the recent population 21

12 Compare two periods locally 22

13 What the comparison stores 23

14 How motion reaches the answer 24

15 A complete numerical walkthrough 25

16 The scientific ideas behind it 26

17 Fixed rules and learned quantities 27

18 Symbol and units reference 28

19 Where the spatial numbers come from 29

20 Where the temporal and scale numbers come from 30

21 References and code provenance 31

1. Start with the actual input

Intensity is a pixel value; movement is a relationship between pixel patterns at different times.

A pixel is one addressable cell of an image. **Grayscale intensity** is its encoded light-dark value. In this input convention, 0 is black, 1 is white and 0.5 is midpoint gray. These numbers are dimensionless. A color pixel contains three component values: red, green and blue, abbreviated RGB. The motion branch takes their arithmetic mean.

$$I_t(x,y) = \frac{X_{t,R}(x,y) + X_{t,G}(x,y) + X_{t,B}(x,y)}{3} \tag{1}$$

X denotes the input color image; R, G and B identify its channels. I denotes the resulting grayscale image. The subscript t selects a frame. Coordinates x and y select a column and row, with x increasing rightward and y downward. Each runs from 0 to 99. Thus $I_5(20,30)=0.8$ means one particular pixel in frame 5 has grayscale value 0.8.

A numerical example

At one pixel, RGB = (0.9, 0.6, 0.3) becomes I = 0.6. The arithmetic conversion is shared at every pixel. Equal-channel gray (0.6, 0.6, 0.6) produces the same motion input. Color information also travels through the separate appearance branch.

$$\mathbf{d} = (x_2 - x_1,\ y_2 - y_1), \qquad \mathbf{v} = \mathbf{d}/\Delta t \tag{2}$$

The bold symbol d is displacement: two numbers describing a position change. If a dot moves from (20,30) to (24,28), d=(4,-2) pixels. The symbol Δt is the elapsed interval, and v is displacement divided by that interval. With two frame steps between the observations, v=(2,-1) pixels per frame step. A negative vertical component means upward movement.

Why a single image is insufficient

A bright dot at (20,30) does not by itself tell us where it came from. A second observation supplies another pattern, and matching the patterns supplies displacement evidence. The computation operates on image arrays; its frame steps acquire physical durations only when a presentation rate is specified.

Source: model.py, forward(), lines 88-96. This document uses t as an integer frame index throughout.

2. Where the model looks

A site is one position on the 10 by 10 analysis grid.

There are 100 sites: 10 rows and 10 columns. Each site has a 24 by 24 grayscale crop containing 576 values. Crops start 10 pixels apart, so nearby sites see some of the same pixels. That shared input is what **overlapping evidence** means.

$$x = 10c - 7 + a, \qquad y = 10r - 7 + b, \qquad i = 10r + c \tag{3}$$

Here r and c are site row and column, each 0 through 9. The single index i labels the site, 0 through 99. Crop coordinates a and b each run from 0 through 23. The crop center is (10c+4.5, 10r+4.5). For site (4,4), the crop covers x=33 through 56 and y=33 through 56. Its neighboring crop at (4,5) covers x=43 through 66.

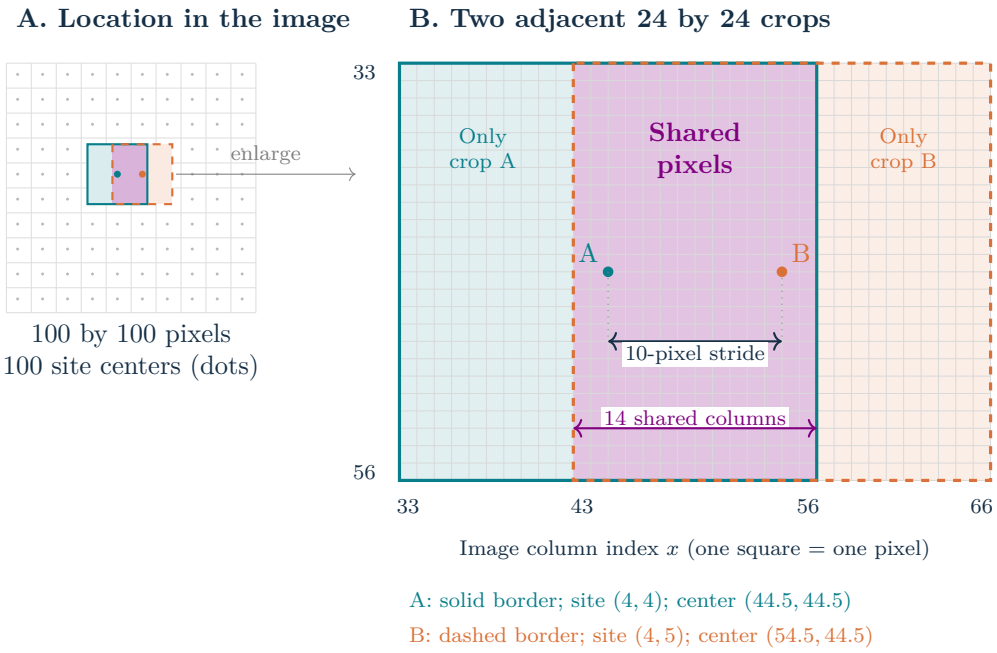


Figure 1. Left: where two neighboring sites lie in the full image. Right: the same crops enlarged to show individual pixels. Crop A covers columns 33-56; crop B covers 43-66. Both cover rows 33-56. Their shared region contains 14 columns by 24 rows = 336 pixels. Their centers are 10 pixels apart.

The model first subtracts 0.5 from the full grayscale image, then pads it with seven zeros on every side. A padded value therefore represents midpoint gray in the original input convention. Padding lets the same crop rule work at the outermost sites.

Why this scale? A larger local neighborhood lets moving structure remain available to matching. A smaller stride makes neighboring measurements overlap. The values 24 and 10 are fixed architectural choices: training adjusts learned weights, but it does not resize these crops or relocate the grid. The attention display assigns each site a 10 by 10 image tile; its motion measurement uses the larger crop.

The first padding: image edges and exact crop coordinates

The 24-pixel crop surrounds a 10-pixel spatial block with seven extra pixels on each side.

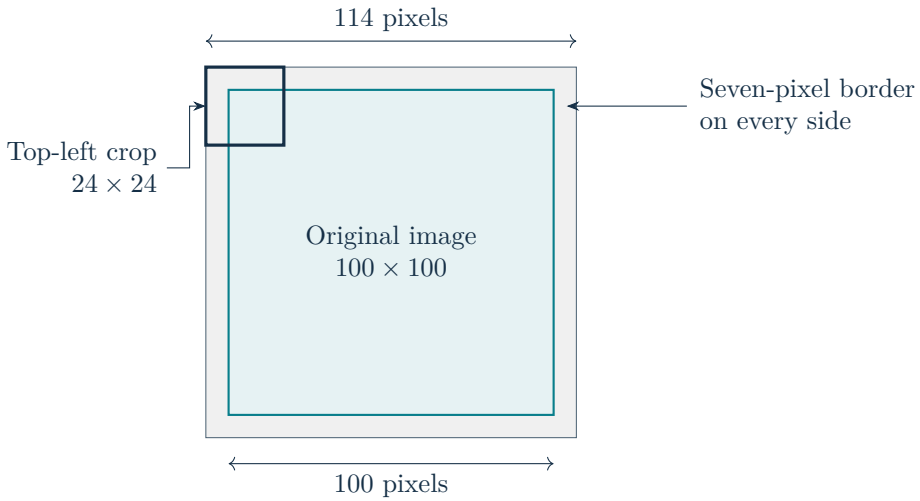
The relation $(24-10)/2 = 7$ explains the border width. Adding seven pixels on each side of the 100-pixel image gives a padded side length of $100 + 7 + 7 = 114$. A crop of width 24 then fits at ten positions separated by a stride of ten:

$$\frac{114-24}{10} + 1 = 10.$$

There are ten rows of sites and ten columns, so 100 crops. Let j be the column of a site and k its row, each from 0 through 9. In original image coordinates, that site's crop covers

$$x = 10j - 7, \ldots, 10j + 16, \qquad y = 10k - 7, \ldots, 10k + 16.$$

The crop's geometric center is $(10j + 4.5, 10k + 4.5)$ in pixel-index coordinates. The top-left crop covers -7 through 16 on each axis. The bottom-right covers 83 through 106. Coordinates outside the original 0 through 99 range receive border values.



What do these zeros mean? The actual order is: subtract 0.5 from the grayscale image, then pad with zero. Thus a border zero corresponds to midpoint gray in original intensity units. Later crop-mean subtraction and windowing operate on these border entries along with the real image entries.

Why local crops? Different parts of an image can move differently. A separate measurement at each site preserves where a motion pattern occurred. The overlap lets neighboring sites share evidence while retaining their separate spatial addresses.

3. Prepare the crop

Remove the local average and gradually reduce the influence of the boundary.

Let $Q_{t,i}(a,b)$ be the crop after the global subtraction of 0.5 and padding. Here a is a local horizontal coordinate and b is a local vertical coordinate; each runs from 0 through 23. The subscript i selects the image site, and t selects the frame. Compute its mean over all 576 entries. Subtracting that mean turns brighter-than-average entries positive and darker-than-average entries negative.

$$\overline{Q}_{t,i} = \frac{1}{576} \sum_{a=0}^{23} \sum_{b=0}^{23} Q_{t,i}(a,b) \tag{4}$$

$$P_{t,i}(a,b) = \left[Q_{t,i}(a,b) - \overline{Q}_{t,i} \right] w(a) w(b) \tag{5}$$

$$w(n) = \frac{1}{2} \left[1 - \cos\left(\frac{2\pi n}{23} \right) \right], \quad n = 0, \ldots, 23 \tag{6}$$

P names the prepared crop. The function w is a nonperiodic Hann window: it is zero at the endpoints and close to one near the center. Multiplying a horizontal weight by a vertical weight gives the two-dimensional weight at each crop pixel. The symbol π is the circle constant; cosine gives the smooth taper.

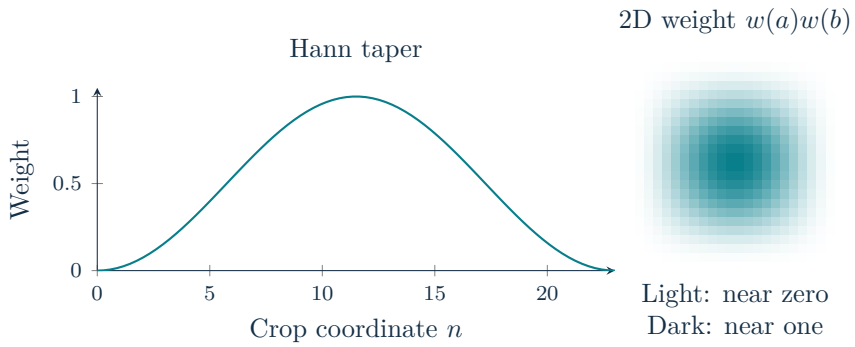


Figure 2. Native vector plots of the one-dimensional Hann taper and its two-dimensional product. Windowing emphasizes the crop interior.

Why subtract the mean? A uniform brightness level would contribute a broad positive match at many shifts. Centering emphasizes variation within the crop. Adding the same constant everywhere in an interior crop leaves the prepared pattern unchanged. At image borders, padding also participates in the mean.

Why taper? A sharply cut edge of a crop can act like an artificial image edge. Windowing emphasizes central structure and makes boundary values fade. Both observations receive a window fixed to the crop coordinates. A moving feature therefore receives slightly different weights as it moves; even a correct match can score below one.

Subtraction and tapering are distinct operations. The crop has zero mean immediately after subtraction; multiplication by the nonuniform window can change that mean again.

4. Ask 81 correspondence questions

For each candidate displacement, multiply aligned values and add the products.

Choose one site and two prepared crops. Call the earlier crop A and the current crop B. A candidate displacement d=(d_x,d_y) says that structure at (a,b) in A should appear at (a+d_x,b+d_y) in B.

Where do -4 and +4 come from? The implementation chooses a search radius R=4 pixels along each axis. R is a fixed hyperparameter: a setting selected when building the model. It specifies which possible shifts the model measures. The code explicitly constructs the integers -4, -3, -2, -1, 0, 1, 2, 3, 4. Minus means left/up, plus means right/down, and zero means no shift along that axis. There are nine choices because both endpoints and zero are included.

$$\#\text{offsets per axis} = 2R + 1, \qquad K = (2R + 1)^2 = 81 \quad \text{when } R = 4 \tag{7}$$

K is the number of horizontal/vertical combinations. Thus **4 is chosen; 9 and 81 are derived**. A radius of 2 would give 25 pairs; a radius of 8 would give 289. A wider range represents larger displacements but expands the retained score population and downstream feature vectors. Radius 4 limits each component of the estimated displacement to [-4,4] for every lag. The crop size determines how much image structure supports a comparison. The search radius independently determines the displacement alternatives represented by that comparison.

$$N(\mathbf{d}) = \sum_{a,b} A(a, b)\, B(a + d_x, b + d_y) \tag{8}$$

N is the unnormalized correlation score. The summation adds products over crop coordinates. For this equation, treat values beyond the 24 by 24 crop as zero. A positive value meeting a positive value contributes positively; two negative values also contribute positively. Opposite signs reduce the score. A displacement that aligns the light-dark structure tends to produce a large sum.

Work through a tiny one-dimensional example

Take an already prepared row A=[0,1,-1,0,0] and a later row B=[0,0,1,-1,0]. At displacement 0, the nonzero product is (-1) times 1, giving N(0)=-1. At displacement +1, the aligned products are 1 times 1 and (-1) times (-1), giving N(+1)=2. The larger score supports a one-pixel shift to the right. The two-dimensional operation performs the same arithmetic in both axes.

$$\mathcal{D} = \{(d_x, d_y) : d_x, d_y \in \{-4, -3, \ldots, 4\}\} \tag{9}$$

The symbol D in this equation is the set of candidate shifts. Arranging the scores as a 9 by 9 table gives a **displacement map**. Its rows indicate vertical shifts and its columns horizontal shifts. It is a map of motion alternatives for one image location. Repeating the calculation at all 100 sites gives 100 such tables for each time lag.

Why keep all scores? A repeating pattern may align at several shifts. A long straight edge can constrain motion across the edge while leaving motion along it ambiguous. Keeping the whole table preserves these alternatives for later computation. Correlation volumes also appear in learned optical-flow systems such as FlowNet [1]. Our score volume is built directly from prepared grayscale crops.

5. Separate shape from signal strength

Normalization measures alignment; a separate factor records how much prepared signal is present.

$$E_A = \sum_{a,b} A(a,b)^2, \quad E_B = \sum_{a,b} B(a,b)^2, \quad z = \sqrt{E_A E_B} \tag{10}$$

$$C(\mathbf{d}) = \frac{N(\mathbf{d})}{\max(z, 10^{-8})}, \qquad r = \frac{z}{z + 0.01} \tag{11}$$

E_A and E_B are sums of squared prepared pixel values, called crop energies. Squaring makes both positive and negative deviations contribute. z is the geometric mean of the two energies. C is the normalized correspondence score. The maximum in its denominator prevents division by a value smaller than 10^{-8}. The scalar r, between zero and one, is the code's energy-based reliability factor.

In the small row example, both energies equal 2, so z=2. The correct-shift score becomes 2/2=1 and the zero-shift score becomes -1/2=-0.5. If A is multiplied by a positive factor a and B by a positive factor b, the product sum and z both scale by ab. Their ratio stays the same when the numerical floor is inactive.

z	**r = z / (z + 0.01)**	**Interpretation**
0	0	No prepared variation in at least one crop.
0.01	0.5	The energy factor gives half weight.
0.09	0.9	Substantial prepared signal.
0.99	0.99	The factor is close to its upper limit.

What exactly is normalized? The denominator uses the energies of the two full prepared crops and is shared across all candidate shifts. Every overlap region is scored using that same full-crop denominator and the original crop preparation. This precise rule matters when reproducing the score.

What does reliability mean here? It measures available prepared-image energy. Rich random noise can have high energy, and a periodic texture can have high energy but several plausible shifts. The displacement table carries match structure; r supplies a separate signal-strength measure. At flat midpoint gray, both C and r are exactly zero.

Code names: costs = C; valid = r. These are similarity scores, so larger means a better match. The name valid denotes energy weighting; the temporal startup rule is described in section 8.

Why energy appears in a motion calculation

The shift-dependent product sum measures correspondence. Energy makes its magnitude interpretable across contrasts.

Consider the prepared rows $A = [0, 1, -1, 0, 0]$ and $B = [0, 0, 1, -1, 0]$. Their shape agrees after a one-pixel shift. Now double every value in A and triple every value in B. The displacement is unchanged, but the correct-shift product sum increases from 2 to 12: each paired product has been multiplied by six.

The squared-value sums increase in a matching way. Initially $E_A = E_B = 2$, so $z = 2$. After the rescaling, $E_A = 8$, $E_B = 18$, and $z = \sqrt{8 \cdot 18} = 12$. Both versions give normalized score $C = 1$ at the correct shift.

Prepared rows	$N(+1)$	E_A	E_B	z	$C(+1)$
Original	2	2	2	2	1
A doubled, B tripled	12	8	18	12	1

The general calculation. If the prepared arrays are multiplied by positive constants a and b, then

$$N' = abN, \quad E'_A = a^2 E_A, \quad E'_B = b^2 E_B, \quad z' = abz, \quad \frac{N'}{z'} = \frac{N}{z}.$$

Primes identify the rescaled quantities. This is why the normalization helps distinguish alignment from contrast. The denominator is identical for all 81 shifts in a crop pair, so dividing by it preserves their ordering.

What energy contributes to motion sensing. A uniform interior patch becomes zero after its own mean is removed. It supplies no pattern whose displacement can be matched. A textured patch supplies positive and negative deviations. Squared deviations quantify the strength of that prepared structure. The separate factor $r = z/(z + 0.01)$ passes this information downstream: strong structure approaches weight one; weak structure approaches weight zero. The constant 0.01 sets the half-weight point.

How brightness changes enter. If an interior crop becomes uniformly brighter by a constant c, its mean also increases by c, giving $(Q + c) - (\overline{Q} + c) = Q - \overline{Q}$. This removes a uniform brightness offset before matching. Positive contrast scaling cancels in C through the calculation above. At image edges, the fixed padding values participate in the mean, so the same algebra applies to a uniform offset of the entire extracted crop, including its border entries.

The correlation table supplies the evidence for *which shift*; r supplies the amount of prepared signal. A high-energy texture can still support several shifts. Those alternatives remain visible in its table.

6. The FFT computes the same sums

We keep the same alignment question and calculate its product sums through a Fourier transform.

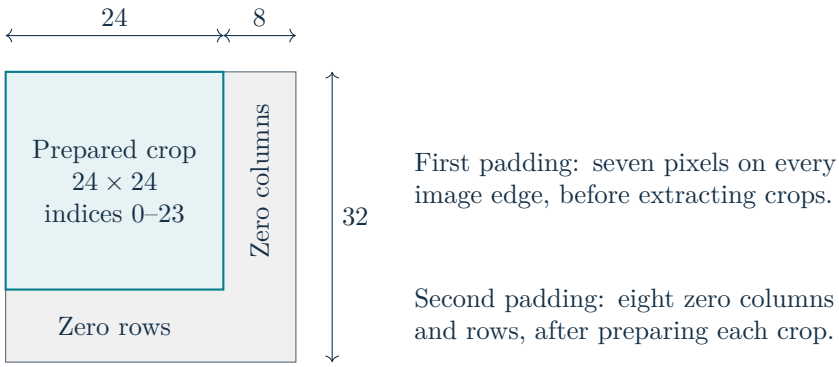
The earlier prepared crop is A and the current prepared crop is B. Both have 24 rows and 24 columns. Before the Fourier calculation, the code extends each to 32×32. Denote these extended arrays by $\widetilde{A}$ and $\widetilde{B}$; the tilde means "with the FFT zeros added."

The second padding: eight columns and eight rows

The prepared crop occupies coordinates 0 through 23 along each axis. New columns 24 through 31 appear on the right; new rows 24 through 31 appear at the bottom. Every new entry is zero. Both crops use the same placement.

$$\widetilde{A}(x, y) = \begin{cases} A(x, y), & 0 \le x, y < 24, \\ 0, & \text{otherwise in the } 32 \times 32 \text{ array.} \end{cases}$$

Here x and y are local horizontal and vertical crop coordinates. Array storage uses row first, so the code stores this mathematical value at row y, column x.



The complete ordering

100×100 image $\longrightarrow$ subtract 0.5 $\longrightarrow$ seven-pixel border $\longrightarrow$ extract 24×24 crops $\longrightarrow$ subtract crop mean $\longrightarrow$ Hann window $\longrightarrow$ extend to 32×32 $\longrightarrow$ FFT.

The first border supplies missing image samples near image edges. The second extension supplies zero prepared signal for the correlation calculation. PyTorch performs the second extension when called with `rfft2(patches, s=(32,32))` [8]. The following pages explain what that transform computes and why the extension prevents false cross-edge matches.

6A. What a Fourier coefficient means

A Fourier transform describes a pattern by how strongly it matches spatial waves.

A spatial wave alternates across position. Frequency counts its cycles across an array; phase describes where its peaks and troughs fall. A broad stripe pattern has a lower spatial frequency than a fine stripe pattern. Combining many frequencies and phases can represent every entry of a finite array exactly.

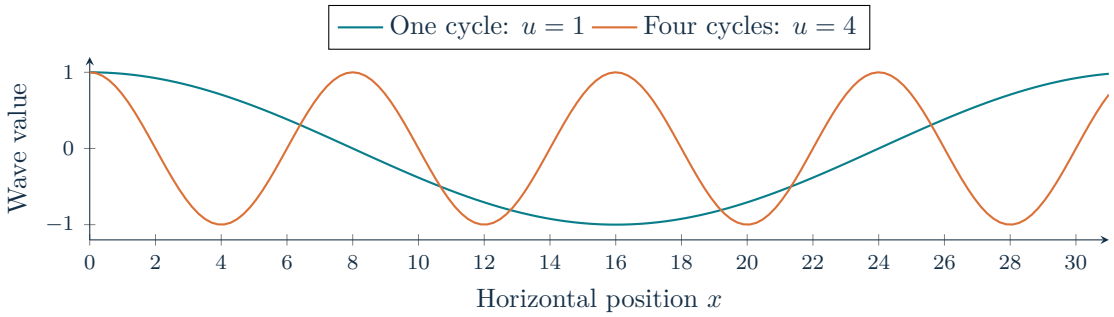
For the equations below, $M = 32$ is the side length of the padded array. The integers u and v select horizontal and vertical frequency indices, each from 0 through $M - 1$. The imaginary unit is written i, with $\mathrm{i}^2 = -1$, to distinguish it from the site index i. Euler's identity states

$$e^{\mathrm{i}\theta} = \cos\theta + \mathrm{i}\sin\theta.$$

Thus a complex exponential is a compact way to keep a cosine and a sine component together. The angle θ is measured in radians. The transform is

$$\widehat{A}(u,v) = \sum_{y=0}^{M-1}\sum_{x=0}^{M-1} \widetilde{A}(x,y)e^{-2\pi\mathrm{i}(ux+vy)/M}. \tag{12}$$

A hat marks a Fourier coefficient. For each frequency pair (u, v), multiply every pixel by a complex wave value at that pixel and add the results. The real part accumulates cosine agreement; the imaginary part accumulates negative sine agreement. Each coefficient is a pair of real numbers, customarily written as one complex number.



At $(u, v) = (0, 0)$ every exponential is one, so the coefficient is the sum of the padded array. Other pairs measure patterns varying horizontally, vertically or in both directions. For a coefficient $a + \mathrm{i}b$, its magnitude is $\sqrt{a^2 + b^2}$ and its phase is its angle in the complex plane. Spatial translation changes phases, which is central to recovering displacement.

6B. Recovering pixels, and a tiny transform by hand

The inverse transform reconstructs each array entry by adding its frequency components:

$$\widetilde{A}(x,y) = \frac{1}{M^2} \sum_{v=0}^{M-1} \sum_{u=0}^{M-1} \widehat{A}(u,v) e^{+2\pi \mathrm{i}(ux+vy)/M}. \tag{13}$$

The forward transform uses a negative exponent. The inverse uses a positive exponent and divides by the number of entries, $M^2 = 1024$. This convention matches the default FFT normalization used in the code. The recovered image values are real; conjugate frequency pairs cancel their imaginary contributions.

A four-entry example. To make the arithmetic visible, use a one-dimensional row $f = [1, 0, -1, 0]$ of length four. Its coefficient at frequency k is

$$F[k] = \sum_{n=0}^{3} f[n] e^{-2\pi \mathrm{i} kn/4}.$$

The index n selects a row entry; k selects a frequency. For $k = 1$, the four exponential multipliers are $[1, -\mathrm{i}, -1, \mathrm{i}]$. Multiplying and adding gives

$$F[1] = 1 \cdot 1 + 0 \cdot (-\mathrm{i}) + (-1) \cdot (-1) + 0 \cdot \mathrm{i} = 2.$$

The complete transform is $F = [0, 2, 0, 2]$. Its nonzero coefficients represent the cosine-shaped pattern in the original row. The inverse reconstructs the entry at $n = 0$ as $(0 + 2 + 0 + 2)/4 = 1$, and the entry at $n = 2$ as $(0 - 2 + 0 - 2)/4 = -1$.

Now translate the row one position right. The shifted row is $g = [0, 1, 0, -1]$, whose transform is $G = [0, -2\mathrm{i}, 0, 2\mathrm{i}]$. Magnitudes remain $[0, 2, 0, 2]$, while phases change. At frequency k, a circular rightward shift by d multiplies the earlier coefficient by

$$e^{-2\pi \mathrm{i} kd/4}.$$

For $d = 1$ and $k = 1$, this factor is $-\mathrm{i}$, converting $F[1] = 2$ into $G[1] = -2\mathrm{i}$. More generally, the shift factor for the two-dimensional arrays is $e^{-2\pi \mathrm{i}(ud_x + vd_y)/M}$.

Fourier coordinates therefore retain positional information through phase. To measure correspondence, the implementation combines the earlier and current coefficients, then transforms back into an array indexed by displacement. The next two pages derive that step.

6C. What makes the FFT fast

The discrete Fourier transform specifies the answer. An FFT is an efficient algorithm for calculating it.

For a one-dimensional array of length M, evaluating all M coefficients separately uses M terms for each coefficient. A radix-two FFT reuses shared calculations by separating even and odd input positions. Let $f[n]$ be an input entry and set $W_M = e^{-2\pi \mathrm{i}/M}$. Then

$$F[k] = \sum_{n=0}^{M-1} f[n] W_M^{kn} \tag{14}$$

$$= \underbrace{\sum_{m=0}^{M/2-1} f[2m] W_{M/2}^{km}}_{E[k]} + W_M^k \underbrace{\sum_{m=0}^{M/2-1} f[2m+1] W_{M/2}^{km}}_{O[k]}. \tag{15}$$

Here m indexes the shorter arrays. E and O are their Fourier transforms, for the even and odd entries respectively. These capital letters are local to this FFT derivation; crop energies elsewhere have subscripts such as E_A.

The same pair of shorter transforms produces two output coefficients:

$$F[k] = E[k] + W_M^k O[k], \qquad F[k + M/2] = E[k] - W_M^k O[k], \quad 0 \le k < M/2.$$

The paired add-and-subtract calculation is often called a butterfly. Repeating the split for length 32 produces lengths $32 \to 16 \to 8 \to 4 \to 2 \to 1$: five combining stages. Each stage processes a number of entries proportional to 32. The general arithmetic growth is therefore proportional to $M \log_2 M$, compared with M^2 for separate direct sums. MIT's FFT notes provide a complementary derivation through polynomial evaluation [7].

How this applies to a crop. The two-dimensional exponential separates into a horizontal factor and a vertical factor. Consequently, a full 32×32 transform can be computed by taking a length-32 transform along every row, then along every column of those results. PyTorch delegates the detailed execution plan to its FFT backend; the factorization explains the arithmetic it implements.

Why the code calls `rfft2`. The prepared input is real. Its coefficients obey the symmetry

$$\widehat{A}((-u) \bmod M, (-v) \bmod M) = \overline{\widehat{A}(u, v)}.$$

An overbar over a complex coefficient means conjugation: $a + \mathrm{i}b$ becomes $a - \mathrm{i}b$. The redundant half can be reconstructed. For a 32×32 real input, `rfft2` stores 32×17 complex coefficients, since $32/2 + 1 = 17$ along the last axis. `irfft2(..., s=(32,32))` restores a 32×32 real result [2,8].

6D. Why multiplying spectra produces correlations

We can derive the correlation identity directly from the forward and inverse definitions.

Write the inverse transform of the elementwise product as $T(d_x, d_y)$:

$$T(d_x, d_y) = \frac{1}{M^2} \sum_{u,v} \overline{\widehat{A}(u,v)}\, \widehat{B}(u,v) e^{2\pi \mathrm{i}(ud_x + vd_y)/M}.$$

The symbol T names the returned correlation table. Each sum in this derivation runs from 0 through $M-1$ for every displayed index. Since the image values are real, conjugating the earlier transform changes the sign of its exponent:

$$\overline{\widehat{A}(u,v)} = \sum_{x,y} \widetilde{A}(x,y) e^{2\pi \mathrm{i}(ux+vy)/M}, \qquad \widehat{B}(u,v) = \sum_{p,q} \widetilde{B}(p,q) e^{-2\pi \mathrm{i}(up+vq)/M}.$$

The coordinates (x, y) index the earlier array; (p, q) independently index the current array. Substitution and regrouping give

$$T(d_x, d_y) = \sum_{x,y,p,q} \widetilde{A}(x,y)\widetilde{B}(p,q) \cdot \left[\frac{1}{M} \sum_{u} e^{2\pi \mathrm{i}u(x-p+d_x)/M}\right] \left[\frac{1}{M} \sum_{v} e^{2\pi \mathrm{i}v(y-q+d_y)/M}\right].$$

The key cancellation. For any integer m, define

$$H(m) = \frac{1}{M} \sum_{k=0}^{M-1} e^{2\pi \mathrm{i}km/M}.$$

If m is a multiple of M, every term is one, so $H(m) = 1$. Otherwise, set $z = e^{2\pi \mathrm{i}m/M}$. The finite geometric sum is $(1 - z^M)/(1 - z)$. Its numerator is zero because $z^M = 1$, while its denominator is nonzero. Thus $H(m) = 0$.

The two brackets therefore keep only coordinates satisfying $p \equiv x + d_x$ and $q \equiv y + d_y$ modulo M. Here the symbol $\equiv$ means equality after wrapping indices by multiples of M. All other coordinate pairs cancel. The result is

$$\boxed{T(d_x, d_y) = \sum_{x,y} \widetilde{A}(x,y)\widetilde{B}((x+d_x) \bmod M, (y+d_y) \bmod M).} \tag{16}$$

This is precisely circular correlation: compare an earlier value with a displaced current value and sum. Conjugating the earlier spectrum establishes the forward displacement convention. Reversing the conjugation would reverse the measured shift sign.

6E. Why the zeros remove wraparound for our search

The Fourier identity produces circular correlation: indices wrap at the transform side length. Padding places zero prepared signal between a crop edge and its periodic neighbor, so the shifts we retain agree with ordinary zero-extended matching.

An explicit false match. Consider toy rows with 24 entries: $A[1] = 1$, $B[22] = 1$, all others zero. At the tested shift $d = -3$, the intended current index is $1 - 3 = -2$, outside the crop. Ordinary zero extension contributes zero. A circular length-24 calculation wraps -2 to 22 and contributes one. Extending to length 32 instead wraps -2 to 30, an appended zero, recovering the intended answer.

Calculation	Accessed position	Product
Zero-extended matching	-2, outside the crop	$1 \cdot 0 = 0$
Circular, length 24	$-2 \bmod 24 = 22$	$1 \cdot 1 = 1$
Circular, length 32	$-2 \bmod 32 = 30$	$1 \cdot 0 = 0$

Why 32 works for every retained shift. Nonzero crop entries occupy indices 0 through 23. Adding a displacement from -4 through $+4$ can access -4 through 27. Indices 0 through 23 retrieve crop values. Indices 24 through 27 retrieve zeros directly. Negative indices -4 through -1 wrap to 28 through 31, also zeros. This proves equality with zero-extended correlation for every requested horizontal and vertical shift.

More generally, for crop side L and tested radius R, choosing $M \geq L + R$ gives enough separation for this argument. Here $24 + 4 = 28$ suffices for the retained range. The implementation uses the convenient power-of-two size 32. Its eight extra positions per axis exceed the required separation of four.

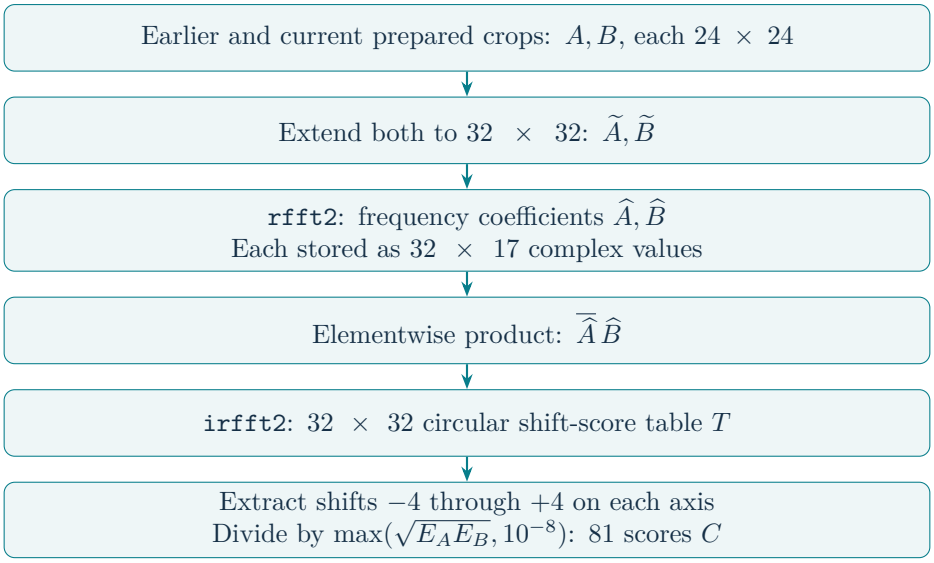
Where the 81 scores live. The inverse output is stored as a 32-row, 32-column table. Its zero-shift entry is at array row 0, column 0. Positive shifts use their ordinary indices; negative shifts use wrapped indices. The code extracts

$$\text{flat_index}(d_x, d_y) = 32(d_y \bmod 32) + (d_x \bmod 32).$$

For example, $(d_x, d_y) = (-1, 2)$ uses row 2, column 31, or flattened index 95. After extraction, the retained values are arranged as a 9×9 displacement table: rows correspond to $d_y = -4, \ldots, 4$, and columns to $d_x = -4, \ldots, 4$. Its center is zero displacement. This table describes motion alternatives at one image site.

6F. The exact calculation, from crop to score table

The Fourier representation is an intermediate coordinate system. The returned scores are indexed by candidate displacement.



The spectral product is elementwise: each frequency pair in the earlier crop is paired with that same frequency pair in the current crop. The inverse transform then combines these contributions at every candidate shift. The derivation on the preceding pages shows why this equals the direct product sums.

The implementation computes a spectrum once for each time and site, then reuses it when comparing different temporal lags. The saved real-input spectrum has 544 complex entries per crop ($32 \cdot 17$). The inverse table has 1024 real entries ($32 \cdot 32$); only 81 are retained for learned processing.

Why return to pixel patterns? The model receives image movies. Comparing their local intensity structure supplies explicit evidence about which displacement aligns observations. Fourier arithmetic calculates that evidence efficiently. The learned refinement, temporal comparisons and shared attention then use the resulting motion features alongside appearance features.

7. Turn scores into a population

A small learned correction adjusts the score map; softmax makes the weights sum to one.

$$\widetilde{C} = C + 0.1\, f_\theta(C), \qquad f_\theta :\ 1 \to 4 \to 1 \text{ channels} \tag{17}$$

C is one 9 by 9 score map. The learned function f_θ applies a 3 by 3 convolution from one channel to four, a GELU nonlinearity, and another 3 by 3 convolution back to one. A convolution forms local weighted sums shared across map positions. Here those positions are **candidate displacements**. A neighboring cell represents a similar possible shift. GELU makes the correction depend nonlinearly on those sums.

The same correction network is used at every image site, time and lag. Both layers use padding one to preserve 9 by 9 shape. The last layer starts with zero weights and bias, so the initial correction is exactly zero. The fixed factor 0.1 scales its contribution. The learned parameters θ can reshape nearby peaks as task learning proceeds.

$$p_k = \frac{\exp(\widetilde{C}_k/\tau_\ell)}{\sum_{j=1}^{81}\exp(\widetilde{C}_j/\tau_\ell)}, \qquad \sum_{k=1}^{81} p_k = 1 \tag{18}$$

k indexes one candidate shift; j indexes all candidates in the denominator. p_k is its normalized weight. The positive number τ_ℓ, called temperature, is separate for each lag ℓ. It starts at 0.04 and is computed as exp of a learned parameter, clamped to [0.015, 0.2]. A smaller temperature magnifies score differences before normalization.

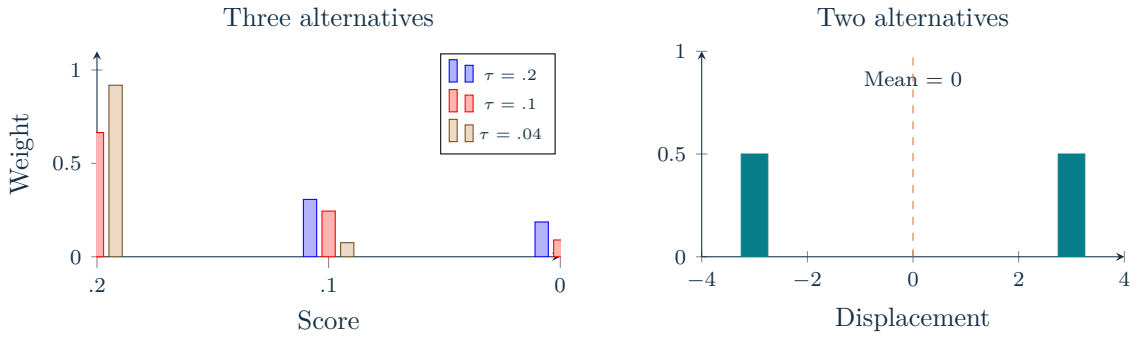


Figure 3. Left: a three-alternative teaching example. Right: a distribution can express two opposing shifts even when its mean is zero.

The odds ratio is $\exp[(\widetilde{C}_i\text{-}\widetilde{C}_j)/\tau]$. A score advantage of 0.1 at τ=0.04 gives about 12.2 times as much weight. The term **population** means the entire 81-number vector, with each coordinate assigned a candidate displacement. These normalized weights can be used as a discrete distribution in subsequent arithmetic.

8. Three lags, one velocity convention

Displacement accumulated over four frames must be divided by four to obtain pixels per frame.

$$s(t, \ell) = \max(t - \ell, 0), \qquad \ell \in \{1, 2, 4\} \tag{19}$$

s is the earlier frame index. The model compares current frame t to frames one, two and four steps earlier. When the requested earlier index would be negative, it uses frame zero. Each lag has its own 81-entry population $p_{t,\ell,i}$. The index i identifies the spatial site.

$$\widehat{\mathbf{d}}_{t,\ell,i} = \sum_{k=1}^{81} p_{t,\ell,i,k}\, \mathbf{d}_k, \qquad \widehat{\mathbf{v}}_{t,\ell,i} = \widehat{\mathbf{d}}_{t,\ell,i}/\ell \tag{20}$$

The hat marks an estimate. d_k is the fixed two-component displacement for candidate k. Multiply each candidate vector by its weight and add: this is the expected displacement. Dividing by the nominal lag gives velocity. For steady rightward motion of one pixel per frame, the expected displacement peaks are (1,0), (2,0) and (4,0). All three imply velocity (1,0).

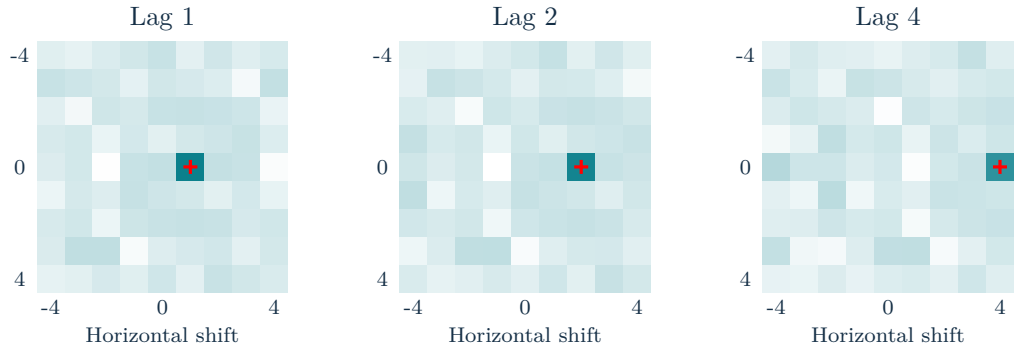


Figure 4. Actual fixed-bank outputs on a seeded synthetic translating texture, at frame 8 and site 44. Red crosses mark the observed maxima. These are input demonstrations, not trained-model results.

Why several lags? A short interval preserves correspondence during fast changes; a longer interval accumulates more displacement for slow movement. Their common displacement range gives different component-wise speed ranges: [-4,4], [-2,2] and [-1,1] pixels per frame. Longer intervals can also lose shared structure as dots disappear or enter the crop.

Startup arithmetic. At t=1, the lag-4 branch compares frame 0 with frame 1 but still divides its estimate by 4. At t=0 all three compare frame 0 with itself. The energy factor has no separate startup mask. These rules are part of the implemented boundary handling and matter when interpreting the first few frames.

The expectation can fall between integer candidates, yielding subpixel values. An equal mixture of (-3,0) and (+3,0) averages to zero; retaining p preserves their distinction from a population concentrated on (0,0).

9. Use motion to predict pixels

The estimate available on the previous frame predicts the current observation.

$$\mathbf{f}_{t,i} = \frac{1}{3} \sum_{\ell \in \{1,2,4\}} \widehat{\mathbf{v}}_{t,\ell,i} \tag{21}$$

f is the compact flow at site i: the simple average of the three velocity estimates. It has two components at each of 100 sites, giving a 2 by 10 by 10 field. The code bilinearly interpolates this field to 2 by 100 by 100. Interpolation forms weighted averages of neighboring site values. It retains pixel-per-frame units; it does not multiply velocities by the grid-size ratio.

$$\widehat{I}_t(x,y) = \text{sample}\left(I_{t-1},\ x - f^x_{t-1}(x,y),\ y - f^y_{t-1}(x,y)\right) \tag{22}$$

$\widehat{I}_t$ is the predicted current image. The superscripts x and y select the horizontal and vertical flow components. The sample operation reads the previous image, interpolating when its coordinates fall between pixels. The flow is from t-1, so this is a one-step continuation prediction: it asks whether the recently estimated motion explains the next image.

Why the minus sign?

If structure moves two pixels right, the value arriving at current position x came from old position x-2. The code visits each output pixel and asks where to read its source. This is called **backward sampling**, even though the supplied vector estimates forward motion. A check with a bright pixel at (40,40) and flow (2,-1) places the output peak at (42,39).

$$g_x = 2\frac{x - f^x}{99} - 1, \qquad g_y = 2\frac{y - f^y}{99} - 1 \tag{23}$$

g_x and g_y are the normalized source coordinates passed to grid_sample: -1 is the first pixel center and +1 the last. Here align_corners=True defines that mapping. The separate flow upsampling uses align_corners=False. Out-of-image sampling uses the nearest border value. The image sampling itself is bilinear [3].

For example, at horizontal coordinate 10.25 the sampler combines columns 10 and 11 with weights 0.75 and 0.25. Applying the analogous rule vertically uses four neighboring pixels. This smoothly varying operation lets a pixel error send gradients back into the motion estimate; differentiable sampling is also the central enabling operation in spatial transformer networks [4].

10. Measure prediction error

The model asks how surprising the next image is relative to the available local signal.

$$e_{t,i} = \frac{1}{100} \sum_{(x,y)\in\Omega_i} [I_t(x,y) - 0.5]^2 \tag{24}$$

Ω_i is the nonoverlapping 10 by 10 image block assigned to site i. Its 100 pixels are different from the larger 24 by 24 motion crop. e measures mean squared deviation from the fixed gray reference 0.5. It retains both brightness offsets and variations inside the block.

$$m_{t,i} = \frac{1}{100} \sum_{(x,y)\in\Omega_i} [\widehat{I}_t(x,y) - I_t(x,y)]^2 \tag{25}$$

$$\epsilon_{t,i} = \frac{m_{t,i}}{\max((e_{t-1,i} + e_{t,i})/2, 10^{-4})} \tag{26}$$

m is mean squared pixel-prediction error. ϵ is that error divided by the average fixed-gray energy of the two frames, with a floor of 10^{-4}. The ratio makes a given error more substantial when the local signal is weak. If both frame energies are 0.04 and m=0.004, ϵ=0.1. The scalar ϵ is supplied to the learned visual features at that site. On frame zero, it is set to zero.

$$\begin{aligned} \omega_{t,i} &= \frac{\sqrt{e_{t-1,i}e_{t,i}}}{\sqrt{e_{t-1,i}e_{t,i}} + 10^{-4}}, \\ L_{\text{pixel}} &= \text{mean}_{t\geq 1,i,\text{batch}}\, \omega_{t,i}\epsilon_{t,i}. \end{aligned} \tag{27}$$

ω is another energy-based weight, this time for the pixel objective. It suppresses contributions from blocks with almost no fixed-gray energy in either frame. The mean averages weighted errors over noninitial frames, image sites and examples. In the preceding numerical example, ω is about 0.9975.

$$L = L_{\text{task}} + 0.05 L_{\text{pixel}} \tag{28}$$

L_{task} is the suite response loss: terminal cross-entropy, with the established value weighting for VDA. The factor 0.05 sets the auxiliary pixel objective's strength. This gives the refinement and temperature a reason to produce useful continuation predictions. Task gradients additionally teach which motion evidence matters for the response. A change, occlusion, dot replacement or new image can all increase the prediction error; its interpretation is learned in context.

11. Remember the recent population

An adaptive trace supports a local comparison on every frame.

Besides predicting the next pixels, the model compares current motion evidence with a running trace. Let $h_{t\text{-}1,\ell,i,k}$ be the value stored just before processing frame t. The trace has one entry for each site, lag and candidate displacement. At the beginning of every trial, every entry is 1/81.

$$\delta_{t,\ell,i} = \sum_{k=1}^{81} |p_{t,\ell,i,k} - h_{t-1,\ell,i,k}| \tag{29}$$

$$h_{t,\ell,i,k} = h_{t-1,\ell,i,k} + \eta r_{t,\ell,i}(p_{t,\ell,i,k} - h_{t-1,\ell,i,k}) \tag{30}$$

δ is a summed absolute difference, also called L1 distance. Two normalized populations have a distance between 0 and 2. η is a learned update fraction obtained by applying sigmoid to one scalar parameter; sigmoid maps any real number into (0,1). η starts at 0.1. Reliability r slows trace updates when prepared signal is weak.

The order matters: compute δ from the **old** trace, use it in this frame's features, then update the trace. If ηr=0.1, the new trace is 90% old trace plus 10% current population. If ηr=0, it remains unchanged.

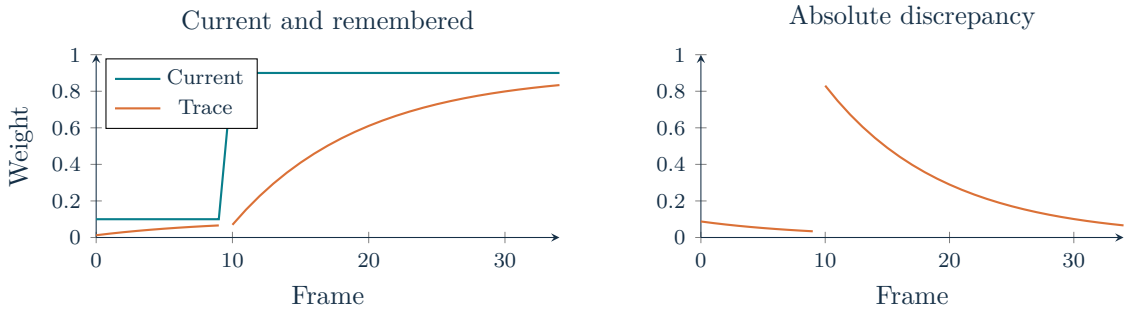


Figure 5. Teaching simulation of one population coordinate, η=0.1 and r=1. The full model adds absolute differences over all 81 coordinates.

After a stable motion changes, the current population moves toward different displacement candidates before the trace has caught up. That difference supplies a transient change signal. Over repeated observations the trace adapts, reducing the discrepancy. With a fixed effective update fraction 0.1, the influence of an old value decays by 0.9 per observation; its half-life is about 6.58 observations.

This trace and the next page's explicit time windows supply different descriptions. The trace has an exponentially fading history. The windows compare two finite, separated periods, preserving a defined earlier baseline for longer changes.

12. Compare two periods locally

The current population is pooled over a recent window and compared with an earlier reference window.

$$\mathcal{W}(t; w, g) = \{\max(0, t + 1 - g - w), \ldots, t - g\} \tag{31}$$

W names the included frame indices. w is the requested window width and g the count of most-recent frames excluded. A window is empty when its ending index is below zero. The recent window uses w=4, g=0; the reference uses w=8, g=8. Thus, once enough frames exist, recent covers t-3 through t and reference covers t-15 through t-8.

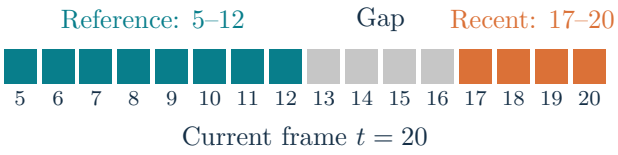


Figure 6. At t=20, reference frames 5-12 and recent frames 17-20 are separated by four unused frames, 13-16. The parameter gap=8 counts excluded recent frames, including the four-frame recent window.

$$\bar{p}^{(w,g)}_{t,\ell,i} = \frac{\sum_{s \in \mathcal{W}} r_{s,\ell,i} p_{s,\ell,i}}{\max(\sum_{s \in \mathcal{W}} r_{s,\ell,i}, 10^{-6})} \tag{32}$$

$$a^{(w,g)}_{t,\ell,i} = \frac{1}{w} \sum_{s \in \mathcal{W}} r_{s,\ell,i} \tag{33}$$

Here s is a frame inside the window. The vector p-bar is a reliability-weighted average of its populations. The scalar a is the total reliability divided by the requested width. It records both signal strength and how much of the window is available. A full window of perfectly reliable observations gives a=1. A half-filled window with the same reliability gives a=0.5.

Name the earlier average R and the recent average O. These letters mean reference and observation. Their joint weight is γ = square root of ($a_{\text{reference}}$ times a_{recent}). γ is zero when the reference has no observations. The first reference observation appears at t=8; it is frame zero. Full reference support appears at t=15. Therefore a seven-frame trial, t=0 through 6, has zero contribution from this explicit comparison branch.

Every quantity uses present or earlier observations. The windows run on every timestep with the same widths and gaps. They receive no cue times or change-event annotations. Cumulative sums implement these weighted boxcar averages efficiently.

13. What the comparison stores

Preserve the old pattern, the new pattern, and their differences at each location.

$$\gamma = \sqrt{a_R a_O}, \quad D_k = |O_k - R_k|, \quad d_1 = \sum_k D_k \tag{34}$$

$$d_{\cos} = 1 - \frac{\sum_k R_k O_k}{\max(\|R\|_2, 10^{-8}) \max(\|O\|_2, 10^{-8})} \tag{35}$$

The site, time and lag indices are suppressed in these equations so the arithmetic is easier to read; all quantities still belong to one site and one lag. D is the 81-entry absolute difference. d_1 is its sum. The L2 norm $||R||_2$ is square root of the sum of squared entries. The cosine compares the directions of the two vectors; subtracting it from one gives a shape difference.

Stored component across 3 lags	Count	Purpose
10γR	243	Earlier local motion alternatives.
10γO	243	Recent local motion alternatives.
10γD	243	Which alternatives changed, coordinate by coordinate.
γd1	3	Overall absolute difference at each lag.
γdcos	3	Population-shape difference at each lag.
Total per site	735	An explicit local comparison vector.

The factor 10 is a fixed feature scale used for the population-valued components. It changes their magnitude before the learned readout; it is an engineering scale in this implementation. The scalar distances do not receive that factor. The same construction is repeated at all 100 image sites.

Why compare before pooling locations?

Suppose a left site initially represents rightward motion and a right site initially represents leftward motion. They then swap. Averaging over locations first gives the same half-rightward, half-leftward population before and after. Comparing each location first exposes a change at both locations. Spatial attention can then select the comparison at the relevant site.

If both window populations are uniform, their difference is zero. If one is concentrated entirely on rightward motion and the other entirely on upward motion, d_1=2 and $d_{\cos}$=1. These are teaching extremes; actual softmax outputs usually distribute weight over multiple alternatives.

14. How motion reaches the answer

The motion pathway feeds both fused visual tokens and explicit local comparisons, selected by the same attention weights.

Input to each visual token	Number of values
Learned appearance responses + raw RGB/RMS statistics	32 + 4
Motion populations scaled by 10r	3 times 81 = 243
Energy reliabilities r	3
Differences δ from the adaptive trace	3
Expected horizontal/vertical velocities at 3 lags	6
Normalized pixel-prediction error ϵ	1
Concatenated input / learned output	292 / 64

$$V_{t,i} = W_{\text{fusion}}\phi_{t,i} + b_{\text{fusion}} + e_i \tag{36}$$

The Greek letter ϕ (phi) names the concatenated 292-entry feature list. W_{fusion} is a learned 64 by 292 matrix; b is its learned 64-entry bias. e_i is a learned position vector identifying site i. V is the resulting 64-entry visual token. Motion and appearance can influence the same token coordinates.

The recurrent attention module scores 100 site keys using one query derived from the previous global CLS memory. A softmax over sites gives weights $\alpha_{t,i}$, which add to one. Each key depends on its current visual token and previous local spatial memory. Consequently, remembered information changes which current site is selected.

$$b_t = \sum_{i=1}^{100} \alpha_{t,i}\, c_{t,i}, \qquad m_t = 0.8m_{t-1} + 0.2b_t \tag{37}$$

$c_{t,i}$ is the 735-entry local comparison from section 13. b_t is its attention-weighted spatial average. m_t is the 735-entry comparison memory, initially zero. The same α weights select learned visual values for the CLS update and modulate spatial memory writes. This is where the motion evidence participates in shared spatial selection.

At the final frame, the 64-entry CLS memory and the 735-entry comparison memory form a 799-entry vector. A learned linear layer and GELU turn it into 64 response features; the selected task head maps those features to answer logits. This separation allows the model to preserve detailed local motion comparisons alongside the compressed fused representation.

15. A complete numerical walkthrough

A synthetic texture moves right by exactly one pixel per frame; the fixed bank is executed unchanged.

The input is a seeded 100 by 100 random grayscale texture, with values between 0.2 and 0.8. Each new frame translates the texture one pixel right. At frame 8, inspect site 44: row 4, column 4. This interior crop stays away from the image wrapping boundary. No trained checkpoint is used; refinement is initially zero and temperature is 0.04.

Lag	Largest-score shift	Score C	r	Mean velocity (x,y)
1	(1, 0)	0.98631	0.99540	(1.0000, 0.0000)
2	(2, 0)	0.94733	0.99548	(1.0000, 0.0000)
4	(4, 0)	0.81390	0.99562	(1.0000, 0.0000)

The peak displacement increases with elapsed time. Dividing by lag returns approximately the same one-pixel-per-frame velocity. The longer-lag score can be lower because the feature has moved farther under the fixed Hann window, and crop boundaries change the overlapping evidence.

Read the computation in sequence

1. At each lag, retrieve the earlier crop and the current crop. Subtract their respective means and apply the same Hann window. **2.** Calculate 81 product sums and divide by the full-crop energy normalization. **3.** Convert the 81 scores into normalized weights with softmax. **4.** Weight the candidate shift coordinates by those values and divide by the lag. **5.** Average the three lag velocities to form the flow used for the next prediction.

6. Compare the current population with the old adaptive trace. **7.** Form the earlier and recent window populations where history exists. **8.** Supply motion features to fusion and supply local comparisons to shared attention. All these operations retain the location until the attention-weighted selection.

Checks executed for this document

The actual CPU motion bank recovers maxima (1,0), (2,0), (4,0). Direct product sums match the FFT scores. A separate impulse-image check verifies the sign of backward sampling. A flat midpoint-gray movie gives zero score and zero reliability. The reference window is empty at t=7, has reliability mass divided by width equal to 1/8 at t=8, and reaches full support at t=15 when every frame is reliable.

All numerical outputs and source hashes are preserved in output/pdf/optic_flow_pathway_verification.json. Synthetic demonstrations illustrate computation, rather than task accuracy.

16. The scientific ideas behind it

Local motion evidence, distributed alternatives and normalization provide the theoretical starting points.

Motion as orientation in space and time [5]

Imagine a horizontal row of image pixels stacked over successive frames. A dot moving right traces a diagonal line in this space-time picture. The slope relates position change to elapsed time. Adelson and Bergen constructed motion-energy models with filters oriented in space-time: paired filter outputs are squared and summed, then opposite directions are compared. Their construction explains how direction selectivity can arise from image signals without assigning an identity to each dot.

Our connection is the underlying use of **spatial relationships across time**. The implemented bank measures alignment at explicit displacements and lags. Its central operation is the product-sum correlation derived above. The energy-model paper supplies a theoretical account of why spatiotemporal structure supports motion sensing.

A population can retain ambiguity [6]

Simoncelli and Heeger modeled responses in visual areas V1 and MT through weighted inputs, rectification and normalization. Their model represents local velocity using a distribution of neuronal responses. A population permits multiple directions and speeds to be expressed together; the pattern across units carries information that a single winning unit would discard.

In our implementation, the 81 coordinates have explicitly assigned displacements. Keeping their joint pattern is useful when the image supports multiple matches. The expected velocity is one summary of that pattern. The larger population remains available for detecting changes even when opposing alternatives cancel in the mean.

Normalization separates two questions

The crop-energy denominator asks how well the prepared structures align relative to their scale. The reliability factor separately carries their signal strength. This is an engineering decomposition motivated by a common computational need: preserve which alternative fits while controlling the effect of input magnitude. The code states exactly how the two quantities are constructed.

Learning uses both prediction and the task

The pixel objective rewards a flow that predicts continuing image structure. The response objective rewards representations useful for the behavioral judgment. Together they can shape refinement and selection. The finite crop, candidate grid, chosen lags and temporal windows specify the problems that learning is asked to solve within this architecture.

17. Fixed rules and learned quantities

The method combines an explicit correspondence calculation with trainable interpretation.

Component	Fixed or learned?	Role
24-pixel crop; stride 10; Hann window	Fixed	Defines local image support.
Lags 1, 2, 4; shifts -4 to +4	Fixed	Defines the measured temporal/spatial alternatives.
FFT correlation and energy normalization	Fixed arithmetic	Produces image-derived match scores.
1 to 4 to 1 convolutional refinement	Learned: 77 parameters	Adjusts local structure in displacement-score space.
Three temperatures	Learned, bounded	Controls sharpness separately at each lag.
Adaptive-trace update fraction	Learned scalar	Controls rate of adaptation.
Window widths/gaps and comparison decay 0.8	Fixed	Specifies explicit temporal comparisons.
Fusion, recurrent attention and response layers	Learned	Uses motion and appearance to solve the task.

The first refinement convolution has 4 times 1 times 3 times 3 weights plus four biases, giving 40 parameters. The second has 1 times 4 times 3 times 3 weights plus one bias, giving 37. Together with three temperatures and one trace-rate parameter, these add 81 learned parameters around the fixed motion computations. Fusion and recurrent processing contain additional learned parameters.

Why these pieces fit together

Raw differences between images mix motion with changes in appearance. Correlation instead evaluates candidate correspondences. The population preserves alternatives; the mean provides a compact displacement. Prediction tests temporal continuation; traces and windows detect changes in evidence. Shared attention identifies which local evidence should influence memory. The task head translates the selected memory into its response vocabulary.

This arrangement builds correspondence and temporal comparison into the representation. Those choices explain the pathway's usefulness and define concrete things to test: shift range versus stimulus speed, crop size versus object size, lag versus dot lifespan, and window support versus trial duration. Each test changes a specific part of the computational chain.

18. Symbol and units reference

The same symbols keep the same roles across this document.

Symbol	Meaning	Shape / units
X; I	RGB input; channel-mean grayscale	3 x 100 x 100; 100 x 100 per frame
t; ℓ; i	Frame index; lag; spatial site	Integer; frame steps; 0-99
x,y; a,b	Image coordinates; crop coordinates	Pixels; integers 0-23
Q; P	Extracted centered crop; prepared crop	24 x 24 each
w; N; C	Hann weight; raw correlation; normalized score	Scalar at each coordinate or shift
E; z; r	Prepared crop energy; joint energy; reliability	Scalar; scalar; fraction in [0,1)
p; τ	Displacement population; temperature	81 weights per site/lag; positive scalar
d; v; f	Displacement; lag velocity; mean-lag flow	2D: pixels; pixels/frame; pixels/frame
I-hat; ϵ; ω	Predicted image; normalized error; loss weight	Image; scalar/site; scalar/site
h; η; δ	Population trace; trace update rate; L1 distance	81 values; fraction; scalar/site/lag
R; O; γ	Reference; recent population; joint support	81 values each; scalar/site/lag
c; α; m	Local comparison; attention; comparison memory	735/site; fraction/site; 735 global
ϕ; V	Concatenated features; fused token	292/site; 64/site

E denotes a sum of prepared squared values on section 5, whereas e on section 10 denotes an average squared deviation from fixed gray over a smaller block. These two energy measurements use different spatial supports and centering rules. The position vector e_i on section 14 is explicitly labeled there; its 64-entry shape distinguishes it from the scalar image energy $e_{t,i}$.

Reading operators: |x| is absolute value; ||v||2 is square root of the sum of squared entries; exp is the exponential; max(a,b) chooses the larger value; a subscript selects an index; a superscript can label a component or window. The arithmetic meaning is specified immediately below each equation.

19. Where the spatial numbers come from

Separate chosen settings, derived counts, and quantities that training adjusts.

A fixed setting specifies the calculation we elected to perform. A derived number follows arithmetically from those settings. A learned value is adjusted by optimization, although its initialization and bounds can still be chosen. Stating these categories makes the method reproducible and makes each design decision available for testing.

Number	Origin	Reason and consequence
100 by 100 input	Chosen input format	The present backbone accepts this resolution. It establishes the coordinate scale for every pixel displacement.
Stride 10; 10 by 10 sites; 100 sites	10 chosen; grid dimensions derived	Stride sets spacing. With the implemented padding and crop size, floor((100+14-24)/10)+1 = 10 sites per axis; 10 squared = 100.
24 by 24 crops	Chosen spatial support	Larger crops supply more correspondence evidence; smaller crops provide more local measurements.
Padding 7 per side	Alignment choice consistent with crop and stride	(24-10)/2=7 aligns crop centers with the 10-pixel blocks. Neighboring crops share 24-10=14 columns.
Radius 4; nine offsets; 81 pairs	4 chosen; 9 and 81 derived	Search extent is limited to four pixels per component. Nine integer offsets include both endpoints and zero; nine squared = 81.
FFT side length 32	Chosen computational size	Power-of-two size above 24. Its eight zero-padded positions per axis suffice for exact finite-crop correlation at retained shifts up to four pixels.
Hann denominator 23	Derived from 24 samples	Indices 0 through 23 place both endpoints at zero in the nonperiodic Hann formula.
Integer shift spacing 1 pixel	Chosen candidate discretization	Softmax expectations can be fractional even though individual candidates lie on this integer grid.

The earlier CorrespondencePredictiveMemory implementation used 50-pixel crops, a 25-pixel stride, FFT side 64 and radius 10, giving 441 displacement pairs. The current values belong to a different local representation. That historical difference documents a change in spatial support and displacement coverage.

20. Where the temporal and scale numbers come from

Their computational effects are explicit; their particular values are design settings.

Setting	Origin	What the setting controls
Lags 1, 2, 4	Chosen frame intervals	Samples different temporal separations. Maximum represented component speed is R divided by lag: 4, 2 and 1 pixels per frame at R=4.
Recent width 4; reference width 8; gap parameter 8	Chosen temporal windows	Compares recent evidence with an older baseline. The four intervening frames follow from 8 excluded recent frames minus the four-frame recent window.
Trace rate initially 0.1	Chosen initialization; learned thereafter	Initial adaptation is 10% per reliable observation. The actual update also multiplies by reliability.
Comparison memory decay 0.8; update 0.2	0.8 chosen; 0.2 = 1-0.8	Retains 80% of the previous comparison memory and adds 20% of the current selected vector.
Temperature initially 0.04; bounds 0.015-0.2	Initialization and bounds chosen; temperature learned	Sets the sensitivity of softmax weights to score differences.
Refinement multiplier 0.1; population multiplier 10	Chosen feature scales	Scales the learned correction and the population features. These coefficients specify the relative magnitudes presented to subsequent computation.
Energy offsets 0.01 and 0.0001	Chosen reliability scales	A ratio z/(z+c) equals one half when z=c. The constants set that half-weight energy scale for two different energy definitions.
Floors 1e-8, 1e-6, 1e-4	Chosen numerical safeguards	Prevent tiny denominators in correlation/cosine, temporal averaging and pixel error. Each floor belongs to its explicitly stated equation.
Pixel loss multiplier 0.05	Chosen objective weight	Sets the contribution of image prediction relative to the behavioral loss.

Together these settings specify the spatial support, temporal support, feature scales and learning objective of the implemented motion pathway.

21. References and code provenance

Primary sources explain the ideas; the inspected source code defines the implemented equations.

[1] Dosovitskiy, A. et al. (2015). FlowNet: Learning Optical Flow with Convolutional Networks. ICCV.

https://arxiv.org/abs/1504.06852

Connection: correlation volumes expose matching evidence to learned processing.

[2] PyTorch. torch.fft.irfft2 documentation.

https://docs.pytorch.org/docs/2.14/generated/torch.fft.irfft2.html

API reference for the real inverse transform. Local numerical checks verify the running implementation.

[3] PyTorch. torch.nn.functional.grid_sample documentation.

https://docs.pytorch.org/docs/2.14/generated/torch.nn.functional.grid_sample.html

API reference for source coordinates, bilinear sampling, border handling and align_corners.

[4] Jaderberg, M., Simonyan, K., Zisserman, A. and Kavukcuoglu, K. (2015). Spatial Transformer Networks. NeurIPS.

https://arxiv.org/abs/1506.02025

Connection: differentiable sampling allows image errors to influence spatial transformations.

[5] Adelson, E. H. and Bergen, J. R. (1985). Spatiotemporal energy models for the perception of motion. JOSA A 2(2), 284-299.

https://doi.org/10.1364/JOSAA.2.000284

Connection: direction-sensitive computations from image structure across space and time.

[6] Simoncelli, E. P. and Heeger, D. J. (1998). A model of neuronal responses in visual area MT. Vision Research 38(5), 743-761.

https://www.cns.nyu.edu/pub/eero/simoncelli96.pdf

Connection: distributed motion representation and normalization in a model of V1/MT.

Implementation record

The original numerical demonstration was verified October 12, 2026 against continuation07/config.json source_sha256; the expanded explanation was compared with the local backbone on October 17, 2026. The actual suite inherits the stabilized backbone through StabilizedAttentionSuite and StabilizedAttentionVDA. Local training uses PyTorch 1.13.1+cu117; the cited API pages are current documentation. The sign, normalization and startup claims were also checked directly on CPU with the local library.

Source entry points: StabilizedAttentionKrauzlis/model.py: LocalMotionBank (lines 14-36), warp (39-45), causal_average (48-60), population/comparison construction (96-114), prediction/trace/fusion (117-141), attention routing (155-171). MemoryGuidedSearch/train.py: update(), task loss plus 0.05 prediction loss. Full paths and SHA256 values are recorded in the companion verification JSON.

Fourier derivation and array APIs

[7] Goemans, M. and Shor, P. (2010). Fast Fourier Transforms. MIT 18.310 lecture notes.

MIT lecture notes: Fast Fourier Transforms

The notes develop the discrete transform, its inverse and an efficient algorithm through polynomial evaluation. Section 6 of this document derives the correlation identity directly for the arrays and shift convention used in our implementation.

[8] PyTorch. `torch.fft.rfft2` documentation.

Real-input two-dimensional FFT: transform size and coefficient storage

The `s=(32,32)` argument requests the transform dimensions. Real-input symmetry permits storing only half of the last frequency axis, including its endpoints.

[9] PyTorch. `torch.nn.functional.pad` documentation.

Constant padding and argument order

The tuple `(7,7,7,7)` adds seven entries left, right, top and bottom to the centered image. The default constant value is zero.

Expanded-edition verification

This edition incorporates the full original pathway explanation and the subsequent questions about brightness, crop geometry, energy, displacement radius, Fourier multiplication and the two padding stages. The padding and Fourier additions were checked on CPU against direct sums. All figures and equations are native LaTeX, TikZ or PGFPlots.

Repository implementation: stabilized motion and attention backbone